



Institut Supérieur de Gestion et d'Administration
ISGA — Casablanca

Rapport de Projet

Système de Gestion de Bibliothèque

Library Management System — .NET 8 Windows Forms

Réalisé par : Abdelilah Zlayji
Filière : 2^{ème} année — Cycle Ingénieur
Encadré par : [Nom de l'encadrant]
Année universitaire : 2025-2026

Casablanca, 1^{er} juin 2026

Remerciements

Je tiens à exprimer ma profonde gratitude à l'ensemble du corps enseignant de l'ISGA Casablanca pour la qualité de la formation reçue. Mes remerciements s'adressent également à mon encadrant pour son accompagnement, ainsi qu'à ma famille pour son soutien constant tout au long de ce projet.

Résumé

Ce rapport présente la conception, le développement et la validation d'un **Système de Gestion de Bibliothèque (SGB)** développé en **C#** avec **.NET 8** et **Windows Forms**. L'application permet la gestion complète des livres, des étudiants et des emprunts, avec authentification sécurisée et tableau de bord statistique. L'architecture repose sur une séparation claire entre les modèles, les services métier (CRUD) et la couche présentation.

Mots-clés : Bibliothèque, .NET 8, Windows Forms, CRUD, UML, SQL, C#.

Table des matières

Remerciements	i
Résumé	ii
1 Introduction Générale	1
1.1 Contexte du projet	1
1.2 Problématique	1
1.3 Objectifs	1
1.3.1 Objectif général	1
1.3.2 Objectifs spécifiques	1
1.4 Méthodologie	2
1.5 Technologies utilisées	2
1.6 Structure du rapport	2
2 Analyse et Conception	3
2.1 Acteurs du système	3
2.2 Diagramme de cas d'utilisation UML (complet)	3
2.3 Description des cas d'utilisation principaux	4
2.4 Diagramme de séquence — Emprunt d'un livre	5
2.5 Diagramme d'activité — Authentification	6
2.6 Modèle Conceptuel de Données (MCD)	7
2.7 Modèle Logique de Données (MLD)	8
3 Réalisation Technique	9
3.1 Architecture de l'application	9

3.2	Modèles C#	9
3.2.1	Classe Livre	9
3.2.2	Classe Etudiant	10
3.2.3	Classe Emprunt	10
3.2.4	Classe Utilisateur	10
3.3	Services CRUD (Couche métier)	10
3.3.1	AuthService — Authentification	11
3.3.2	LivreService — CRUD Livres	11
3.3.3	EtudiantService — CRUD Étudiants	12
3.3.4	EmpruntService — Gestion des emprunts	12
3.4	Point d'entrée de l'application	13
3.5	Structure du projet	13
4	Interfaces Utilisateur	14
4.1	Présentation de l'interface	14
4.2	Formulaire de connexion (LoginForm)	14
4.3	Tableau de bord (DashboardForm)	15
4.4	Gestion des livres (LivresForm)	16
4.5	Gestion des étudiants (EtudiantsForm)	17
4.6	Gestion des emprunts (EmpruntsForm)	17
4.7	Navigation entre les écrans	18
5	Tests Fonctionnels Détaillés	19
5.1	Stratégie de test	19
5.2	Tests d'authentification	19
5.3	Tests CRUD Livres	19
5.4	Tests CRUD Étudiants	20
5.5	Tests Emprunts	20
5.6	Synthèse des résultats	21
6	Conclusion Générale et Perspectives	22
6.1	Bilan du projet	22
6.2	Apports personnels	22
6.3	Limites actuelles	22
6.4	Perspectives d'évolution	23
6.5	Conclusion finale	23
A	Code source complet des modèles C#	24
A.1	Livre.cs	24
A.2	Etudiant.cs	24
A.3	Emprunt.cs	25
A.4	Utilisateur.cs	25
B	Captures d'écran supplémentaires	26
C	Bibliographie et Webographie	29

Table des figures

2.1	Diagramme de cas d'utilisation complet — SGB	4
2.2	Diagramme de séquence — Processus d'emprunt	5
2.3	Diagramme d'activité — Processus d'authentification	6
2.4	Modèle Conceptuel de Données (MCD) — Notation Merise	7
2.5	Modèle Logique de Données (MLD)	8
4.1	Formulaire de connexion — LoginForm	15
4.2	Tableau de bord avec statistiques en temps réel	16
4.3	Interface CRUD — Gestion des livres	16
4.4	Interface CRUD — Gestion des étudiants	17
4.5	Interface — Gestion des emprunts et retours	17
4.6	Flux de navigation entre les interfaces	18
B.1	Annexe — Connexion	26
B.2	Annexe — Dashboard	27
B.3	Annexe — Livres	27
B.4	Annexe — Étudiants	28
B.5	Annexe — Emprunts	28

Liste des tableaux

1.1	Stack technologique du projet	2
2.1	Acteurs identifiés	3
2.3	Dictionnaire de données — Tables principales	8
3.1	Correspondance architecture	9
5.5	Bilan des tests fonctionnels	21
6.1	Roadmap des améliorations futures	23

Chapitre 1

Introduction Générale

1.1 Contexte du projet

La gestion d'une bibliothèque universitaire ou scolaire implique le suivi rigoureux des ouvrages, des usagers (étudiants) et des opérations d'emprunt et de retour. Les méthodes manuelles (registres papier, fichiers Excel) sont sources d'erreurs, de pertes d'information et de lenteur opérationnelle.

Ce projet répond à ce besoin en proposant une application desktop moderne, intuitive et performante, développée dans le cadre du cycle ingénieur à l'**ISGA Casablanca**.

1.2 Problématique

Comment concevoir et développer une application informatique permettant à un administrateur de bibliothèque de :

- gérer le catalogue de livres (ajout, modification, suppression, recherche) ;
- gérer les étudiants inscrits ;
- enregistrer les emprunts et retours de livres ;
- visualiser des statistiques en temps réel ;
- sécuriser l'accès par authentification ?

1.3 Objectifs

1.3.1 Objectif général

Concevoir et implémenter un **Système de Gestion de Bibliothèque (SGB)** complet, fonctionnel et ergonomique.

1.3.2 Objectifs spécifiques

1. Modéliser le système à l'aide de diagrammes UML (cas d'utilisation, classes, séquence, activité, déploiement).
2. Concevoir la base de données (MCD et MLD).
3. Développer les modèles C# et les services CRUD.

4. Créer une interface graphique professionnelle avec Windows Forms.
5. Valider le système par des tests fonctionnels détaillés.

1.4 Méthodologie

Le projet suit une approche itérative :

- **Analyse** : recueil des besoins, diagrammes UML, modélisation des données.
- **Conception** : architecture en couches (Modèles / Services / Formulaires).
- **Développement** : implémentation en C# .NET 8.
- **Tests** : validation fonctionnelle scénario par scénario.
- **Rédaction** : documentation technique et rapport.

1.5 Technologies utilisées

TABLE 1.1 – Stack technologique du projet

Composant	Technologie
Langage	C# 12
Framework	.NET 8.0 (Windows Forms)
IDE	Visual Studio 2022
Modélisation	UML, Merise (MCD/MLD)
Base de données	SQL (MySQL / SQL Server)
Architecture	Couches (Models, Services, Forms)

1.6 Structure du rapport

Ce rapport est organisé en six chapitres :

- **Chapitre 1** : Introduction générale (présent chapitre).
- **Chapitre 2** : Analyse et conception (UML, MCD, MLD).
- **Chapitre 3** : Réalisation technique (SQL, modèles C#, services CRUD).
- **Chapitre 4** : Interfaces utilisateur (captures d'écran).
- **Chapitre 5** : Tests fonctionnels détaillés.
- **Chapitre 6** : Conclusion générale et perspectives.

Conclusion du chapitre

Ce chapitre a posé le cadre du projet SGB en définissant le contexte, la problématique, les objectifs et la méthodologie adoptée. L'application vise à digitaliser la gestion bibliothécaire via une solution .NET 8 Windows Forms structurée en couches. Les chapitres suivants détailleront la modélisation UML, l'implémentation technique et la validation du système.

Chapitre 2

Analyse et Conception

2.1 Acteurs du système

TABLE 2.1 – Acteurs identifiés

Acteur	Description
Administrateur	Utilisateur authentifié (admin/admin123) gérant l'ensemble du système
Système	Composant logiciel exécutant les règles métier et la persistance

2.2 Diagramme de cas d'utilisation UML (complet)

La figure 2.1 présente l'ensemble des cas d'utilisation du SGB.

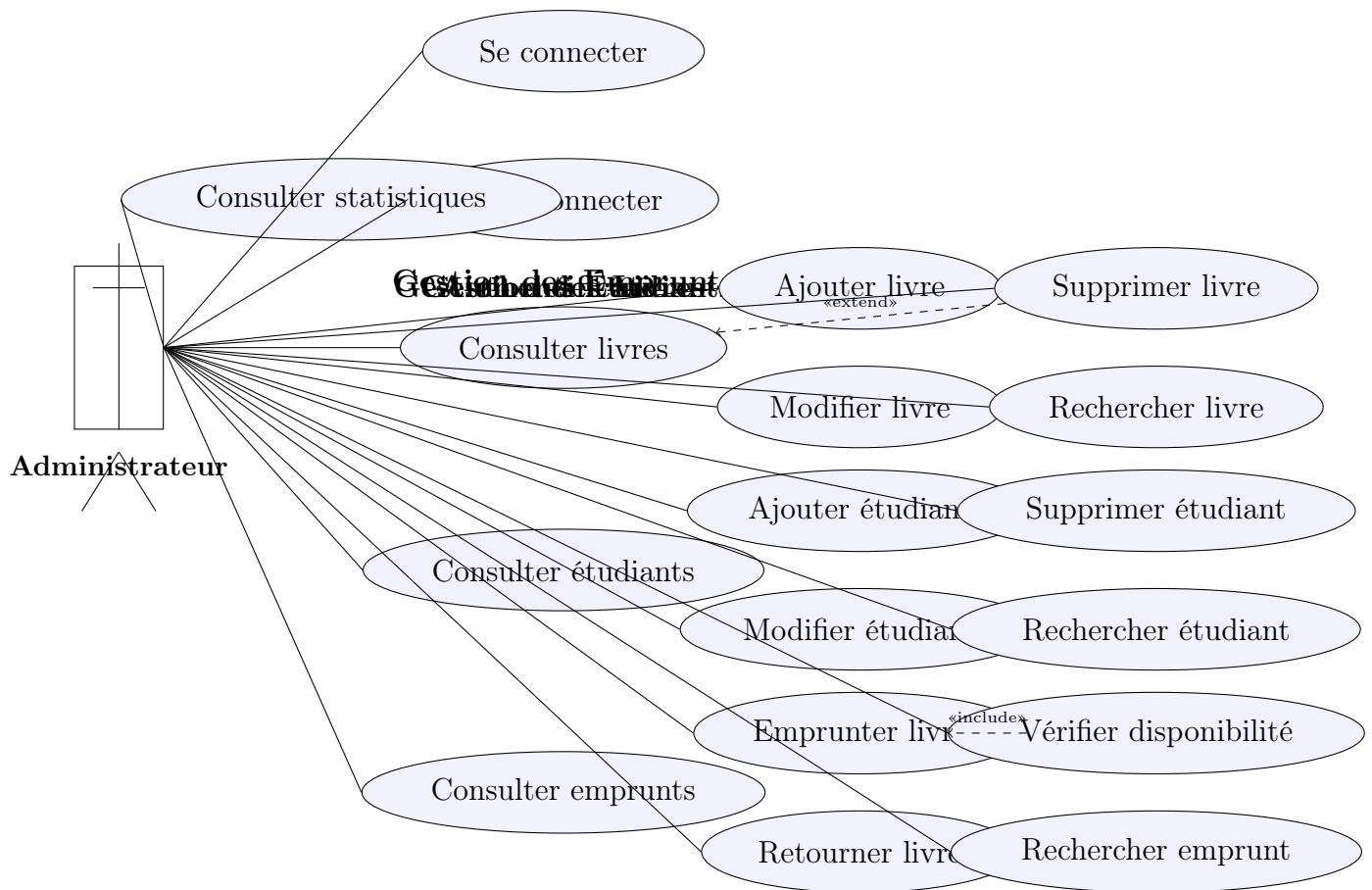


FIGURE 2.1 – Diagramme de cas d'utilisation complet — SGB

2.3 Description des cas d'utilisation principaux

Cas d'utilisation	Acteur	Description
Se connecter	Administrateur	Saisie username/password, validation via AuthService
Gérer livres	Administrateur	CRUD complet avec recherche en temps réel
Gérer étudiants	Administrateur	CRUD avec validation email et téléphone
Emprunter livre	Administrateur	Sélection livre/étudiant, vérification disponibilité
Retourner livre	Administrateur	Mise à jour DateRetour, libération exemplaire
Consulter statistiques	Administrateur	Affichage nb livres, étudiants, emprunts en cours

2.4 Diagramme de séquence — Emprunt d'un livre

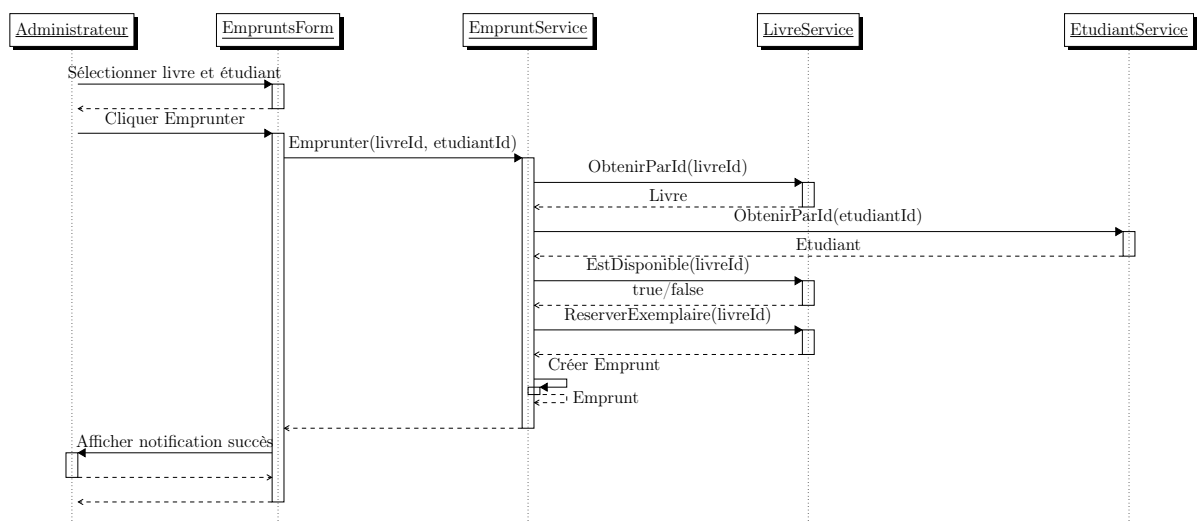


FIGURE 2.2 – Diagramme de séquence — Processus d'emprunt

2.5 Diagramme d'activité — Authentification

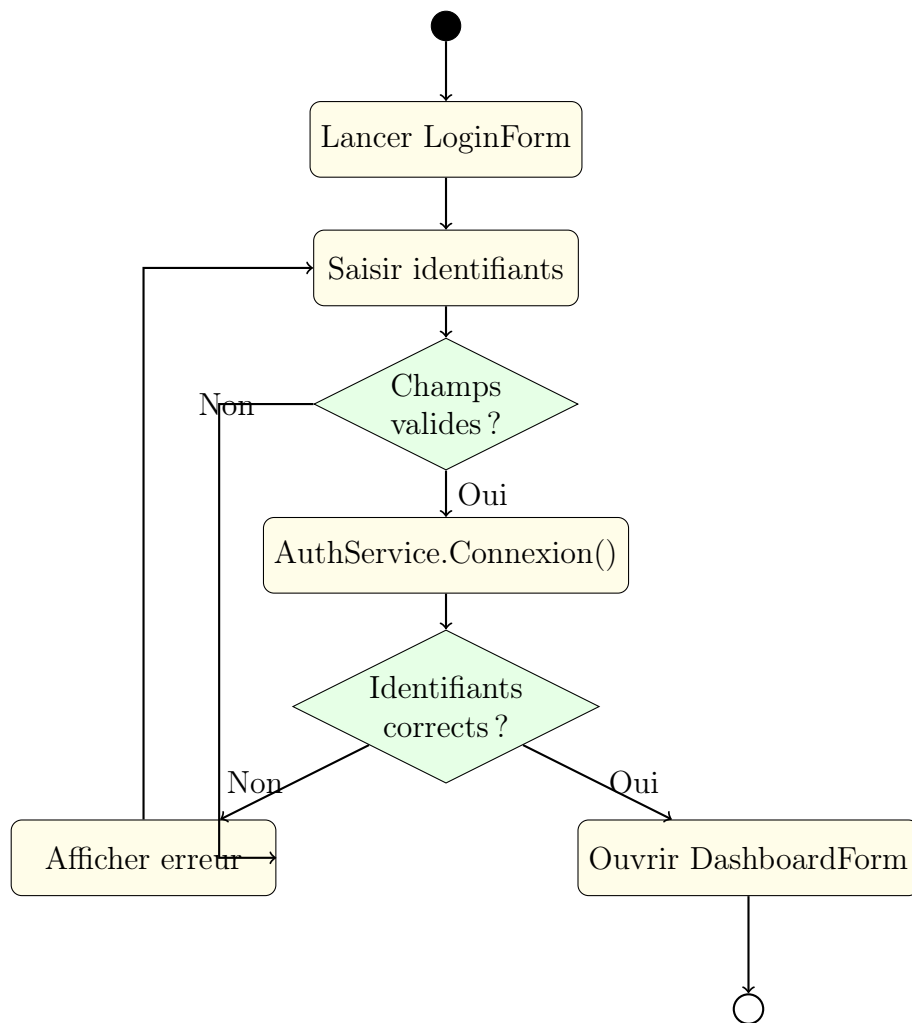


FIGURE 2.3 – Diagramme d'activité — Processus d'authentification

2.6 Modèle Conceptuel de Données (MCD)

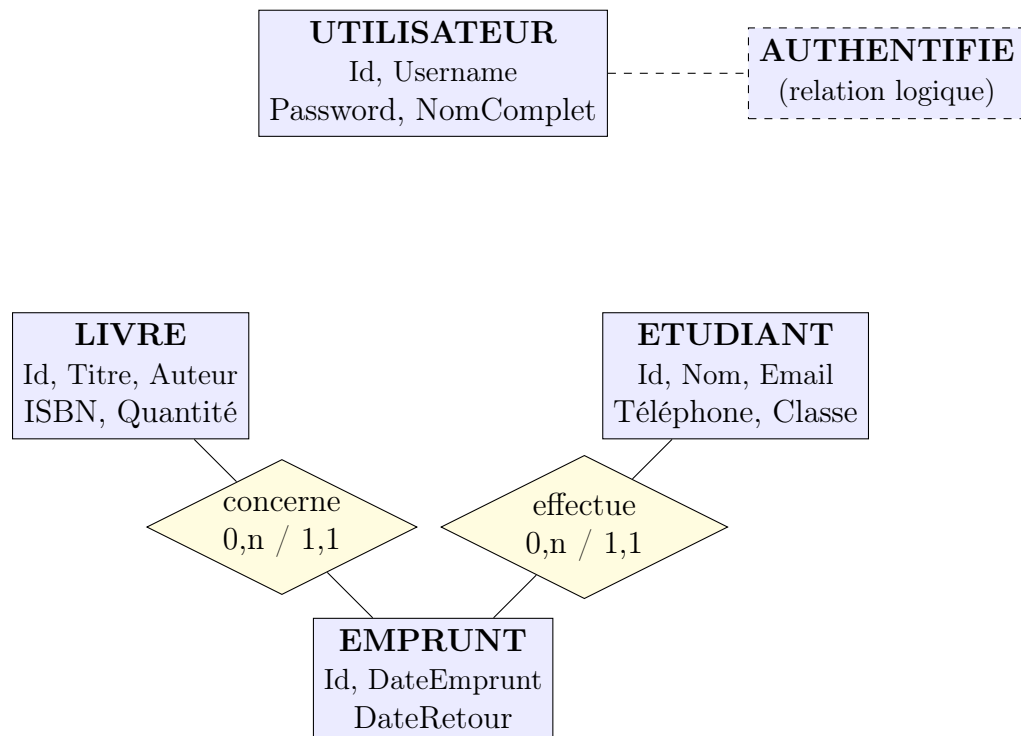


FIGURE 2.4 – Modèle Conceptuel de Données (MCD) — Notation Merise

Règles de gestion :

- Un **LIVRE** peut être concerné par 0 à n **EMPRUNTS**.
- Un **ETUDIANT** effectue 0 à n **EMPRUNTS**.
- Chaque **EMPRUNT** concerne exactement un livre et un étudiant.
- Un livre ne peut être supprimé si des exemplaires sont empruntés.

2.7 Modèle Logique de Données (MLD)

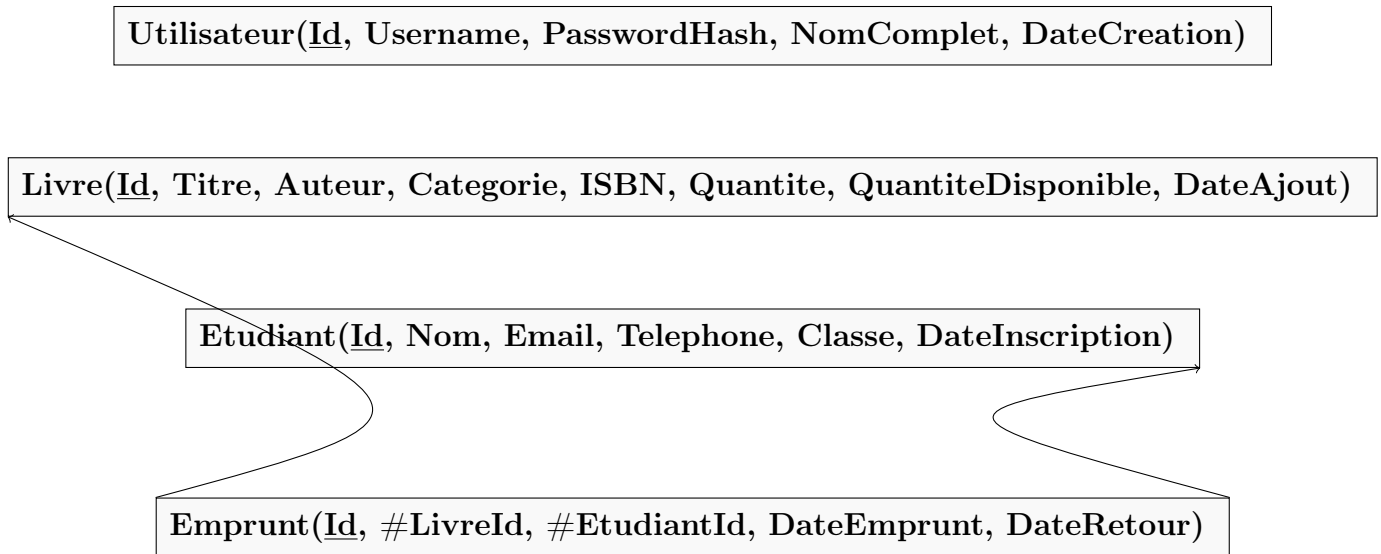


FIGURE 2.5 – Modèle Logique de Données (MLD)

TABLE 2.3 – Dictionnaire de données — Tables principales

Table	Attribut	Type	Contrainte
Livre	ISBN	VARCHAR(20)	UNIQUE, NOT NULL
Livre	QuantiteDisponible	INT	$\leq$ Quantite
Etudiant	Email	VARCHAR(150)	UNIQUE, NOT NULL
Emprunt	LivreId	INT	FK $\rightarrow$ Livre(Id)
Emprunt	EtudiantId	INT	FK $\rightarrow$ Etudiant(Id)

Conclusion du chapitre

Ce chapitre a établi les fondations conceptuelles du SGB à travers une modélisation UML complète (cas d'utilisation, classes, séquence, activité, déploiement) et une modélisation de données Merise (MCD et MLD). Ces artefacts guident l'implémentation technique présentée au chapitre suivant et garantissent la cohérence entre les besoins fonctionnels et la structure du code C#.

Chapitre 3

Réalisation Technique

3.1 Architecture de l'application

L'application suit une architecture en trois couches :

TABLE 3.1 – Correspondance architecture

Couche	Dossier	Rôle
Présentation	Forms/	Interface utilisateur (équivalent Vues MVC)
Métier	Services/	Logique CRUD (équivalent Contrôleurs)
Données	Models/	Entités métier
Transversal	Assets/	Thème UI (UITheme)

3.2 Modèles C#

3.2.1 Classe Livre

```
1 namespace LibraryManagementSystem.Models;
2
3 public class Livre
4 {
5     public int Id { get; set; }
6     public string Titre { get; set; } = string.Empty;
7     public string Auteur { get; set; } = string.Empty;
8     public string Categorie { get; set; } = string.Empty;
9     public string ISBN { get; set; } = string.Empty;
10    public int Quantite { get; set; }
11    public int QuantiteDisponible { get; set; }
12
13    public override string ToString() => $"{Titre} - {Auteur}";
14 }
```

Listing 3.1 – Models/Livre.cs

3.2.2 Classe Etudiant

```
1 public class Etudiant
2 {
3     public int Id { get; set; }
4     public string Nom { get; set; } = string.Empty;
5     public string Email { get; set; } = string.Empty;
6     public string Telephone { get; set; } = string.Empty;
7     public string Classe { get; set; } = string.Empty;
8
9     public override string ToString() => Nom;
10 }
```

Listing 3.2 – Models/Etudiant.cs

3.2.3 Classe Emprunt

```
1 public class Emprunt
2 {
3     public int Id { get; set; }
4     public Livre Livre { get; set; } = new();
5     public Etudiant Etudiant { get; set; } = new();
6     public DateTime DateEmprunt { get; set; }
7     public DateTime? DateRetour { get; set; }
8
9     public bool EstRetourne => DateRetour.HasValue;
10    public string Statut => EstRetourne ? "Retourn " : "En cours";
11 }
```

Listing 3.3 – Models/Emprunt.cs

3.2.4 Classe Utilisateur

```
1 public class Utilisateur
2 {
3     public string Username { get; set; } = string.Empty;
4     public string Password { get; set; } = string.Empty;
5     public string NomCompleet { get; set; } = string.Empty;
6 }
```

Listing 3.4 – Models/Utilisateur.cs

3.3 Services CRUD (Couche métier)

Les services implémentent les opérations CRUD et remplissent le rôle des contrôleurs dans une architecture MVC.

3.3.1 AuthService — Authentification

```
1 public class AuthService
2 {
3     public static Utilisateur? UtilisateurConnecte { get; private set
4         ; }
5
6     public bool Connexion(string username, string password)
7     {
8         var user = Utilisateurs.Find(u =>
9             u.Username.Equals(username.Trim(),
10                StringComparison.OrdinalIgnoreCase) &&
11                u.Password == password);
12
13         if (user == null) return false;
14         UtilisateurConnecte = user;
15         return true;
16     }
17
18     public void Deconnexion() => UtilisateurConnecte = null;
19 }
```

Listing 3.5 – Services/AuthService.cs (extrait)

3.3.2 LivreService — CRUD Livres

```
1 public class LivreService
2 {
3     public List<Livre> ObtenirTous() => Livres.ToList();
4
5     public Livre Ajouter(Livre livre)
6     {
7         livre.Id = _nextId++;
8         livre.QuantiteDisponible = livre.Quantite;
9         Livres.Add(livre);
10        return livre;
11    }
12
13    public bool Modifier(Livre livre) { /* ... */ return true; }
14
15    public bool Supprimer(int id)
16    {
17        var livre = ObtenirParId(id);
18        if (livre.Quantite != livre.QuantiteDisponible)
19            throw new InvalidOperationException(
20                "Impossible de supprimer : exemplaires empruntés.");
21        return Livres.Remove(livre);
22    }
23
24    public List<Livre> Rechercher(string terme) { /* ... */ }
25 }
```


Listing 3.6 – Services/LivreService.cs (CRUD complet)

3.3.3 EtudiantService — CRUD Étudiants

```
1 public class EtudiantService
2 {
3     public Etudiant Ajouter(Etudiant etudiant)
4     {
5         etudiant.Id = _nextId++;
6         Etudiants.Add(etudiant);
7         return etudiant;
8     }
9
10    public bool Modifier(Etudiant etudiant) { /* ... */ }
11    public bool Supprimer(int id) =>
12        Etudiants.RemoveAll(e => e.Id == id) > 0;
13 }
```

Listing 3.7 – Services/EtudiantService.cs (CRUD)

3.3.4 EmpruntService — Gestion des emprunts

```
1 public Emprunt Emprunter(int livreId, int etudiantId)
2 {
3     if (!_livreService.EstDisponible(livreId))
4         throw new InvalidOperationException("Livre non disponible.");
5
6     _livreService.ReserverExemplaire(livreId);
7
8     var emprunt = new Emprunt
9     {
10         Id = _nextId++,
11         Livre = livre,
12         Etudiant = etudiant,
13         DateEmprunt = DateTime.Now
14     };
15     Emprunts.Add(emprunt);
16     return emprunt;
17 }
18
19 public bool Retourner(int empruntId)
20 {
21     emprunt.DateRetour = DateTime.Now;
22     _livreService.LibererExemplaire(emprunt.Livre.Id);
23     return true;
24 }
```

Listing 3.8 – Services/EmpruntService.cs

3.4 Point d'entrée de l'application

```
1 internal static class Program
2 {
3     [STAThread]
4     private static void Main()
5     {
6         ApplicationConfiguration.Initialize();
7         Application.Run(new LoginForm());
8     }
9 }
```

Listing 3.9 – Program.cs

3.5 Structure du projet

```
LibraryManagementSystem/
  Models/           (Livre, Etudiant, Emprunt, Utilisateur)
  Services/         (AuthService, LivreService, EtudiantService, EmpruntService)
  Forms/            (LoginForm, DashboardForm, LivresForm, ...)
  Assets/           (UITheme.cs, images/, icons/)
  Program.cs
```

Conclusion du chapitre

Ce chapitre a détaillé l'implémentation technique du SGB : script SQL, modèles C#, services CRUD et point d'entrée. L'architecture en couches assure la maintenabilité et la séparation des responsabilités. La couche présentation (formulaire Windows Forms) est documentée au chapitre suivant.

Chapitre 4

Interfaces Utilisateur

4.1 Présentation de l'interface

L'application utilise **Windows Forms** avec un thème visuel personnalisé (`UITheme.cs`) :

- Couleurs : bleu foncé `#1E3A5F`, bleu clair `#4F9DDE`
- Police : Segoe UI
- Composants stylisés : `DataGridView`, boutons arrondis, notifications toast

4.2 Formulaire de connexion (LoginForm)

Interface d'authentification avec validation des champs. Identifiants par défaut : `admin / admin123`.

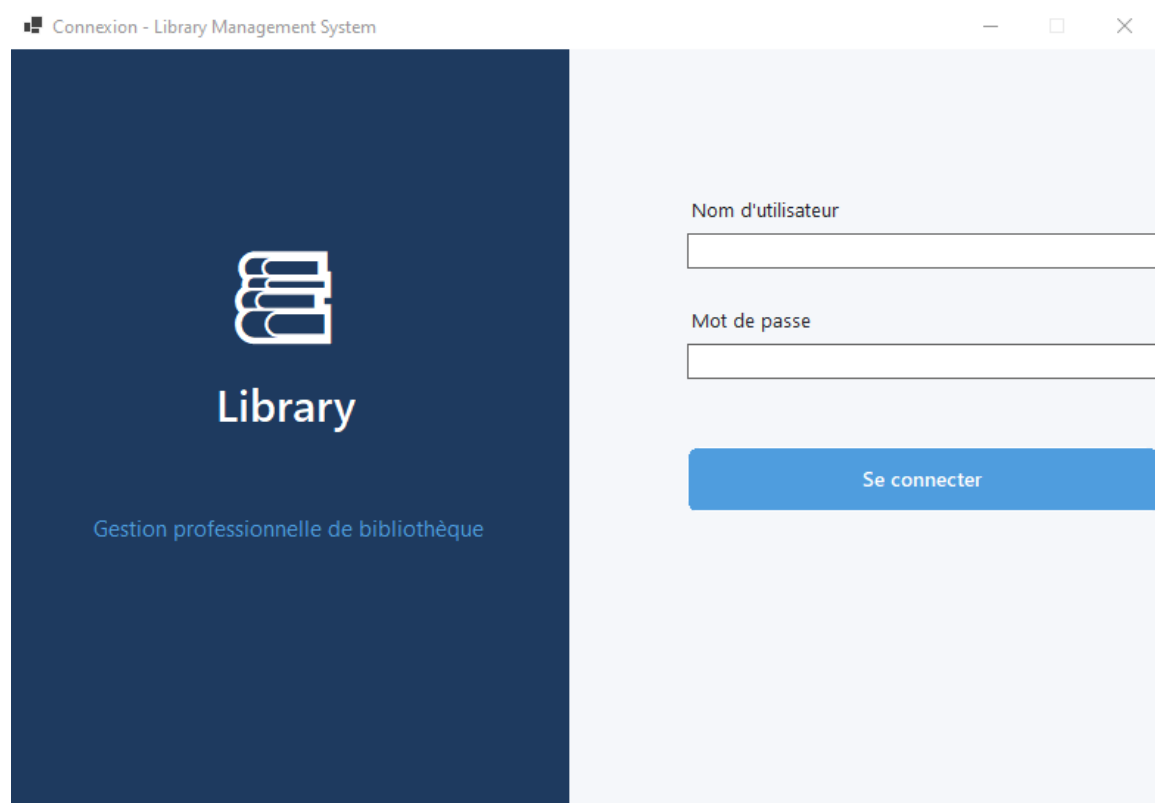


FIGURE 4.1 – Formulaire de connexion — LoginForm

Fonctionnalités :

- Validation des champs vides
- Connexion via AuthService
- Touche Entrée pour valider
- Messages d'erreur contextuels

4.3 Tableau de bord (DashboardForm)

Hub central avec sidebar de navigation et cartes statistiques.

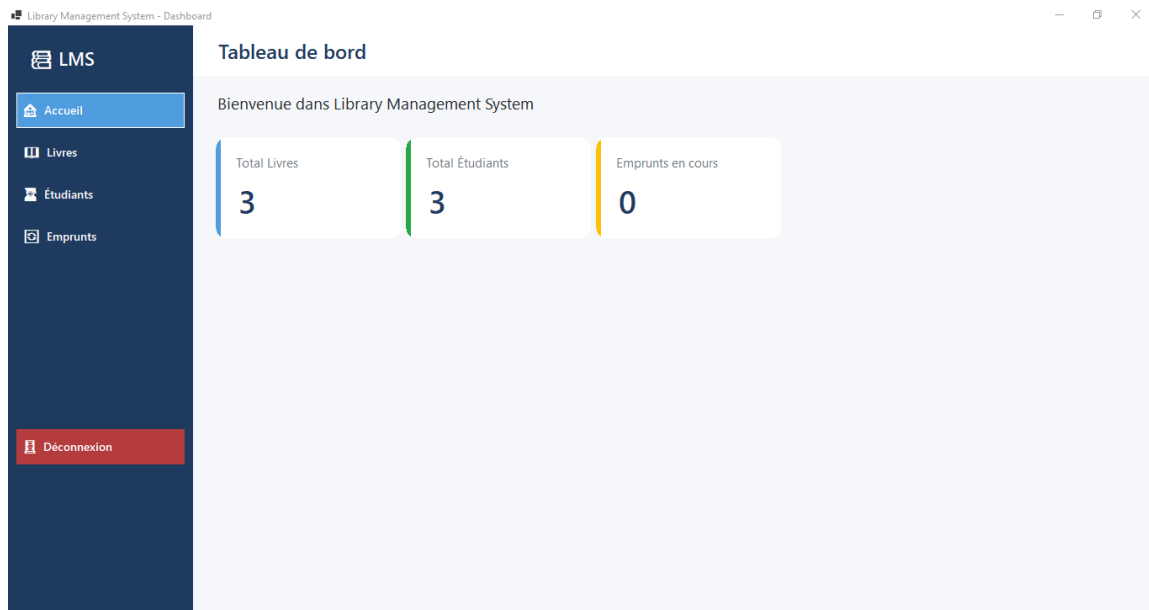


FIGURE 4.2 – Tableau de bord avec statistiques en temps réel

Statistiques affichées :

Carte	Source
Nombre de livres	LivreService.CompterTitres()
Nombre d'étudiants	EtudiantService.CompterTotal()
Emprunts en cours	EmpruntService.CompterEnCours()

4.4 Gestion des livres (LivresForm)

Interface CRUD complète avec DataGridView et formulaire de saisie.

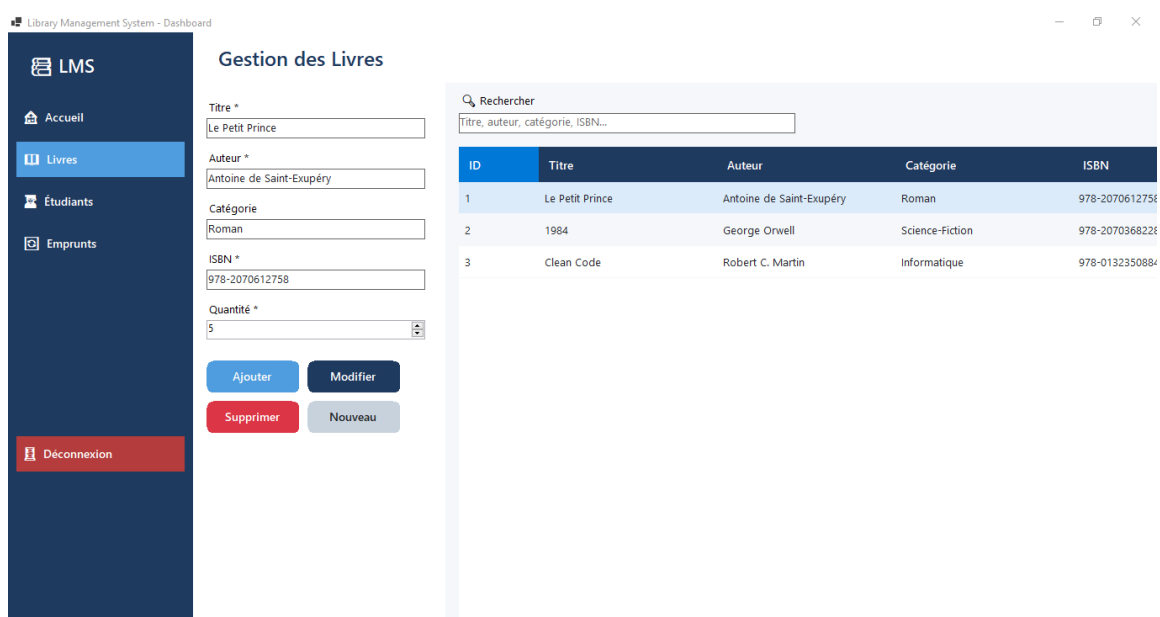


FIGURE 4.3 – Interface CRUD — Gestion des livres

Actions disponibles : Ajouter, Modifier, Supprimer, Nouveau, Recherche en temps réel.

4.5 Gestion des étudiants (EtudiantsForm)

Gestion des Étudiants

Nom *
Dupont Jean

Email *
jean.dupont@email.com

Téléphone *
0612345678

Classe
L3 Informatique

Ajouter Modifier
Supprimer Nouveau

Rechercher
Nom, email, téléphone, classe...

ID	Nom	Email	Téléphone
1	Dupont Jean	jean.dupont@email.com	0612345678
2	Martin Sophie	sophie.martin@email.com	0698765432
3	Bernard Lucas	lucas.bernard@email.com	0655443322

FIGURE 4.4 – Interface CRUD — Gestion des étudiants

Validation : nom obligatoire, email avec @, téléphone obligatoire.

4.6 Gestion des emprunts (EmpruntsForm)

Gestion des Emprunts

Livre
Le Petit Prince

Disponible (3/5)

Étudiant
Dupont Jean

Emprunter Retourner
Actualiser

Rechercher
Livre, étudiant, statut...

ID	Livre	Étudiant	Date emprunt
1	Le Petit Prince	Dupont Jean	01-06-2026
2	Le Petit Prince	Dupont Jean	01-06-2026

Emprunt enregistré avec succès.

FIGURE 4.5 – Interface — Gestion des emprunts et retours

Fonctionnalités :

- Sélection livre/étudiant via ComboBox
- Indicateur de disponibilité en temps réel
- Enregistrement emprunt et retour
- Recherche par titre, étudiant ou statut

4.7 Navigation entre les écrans

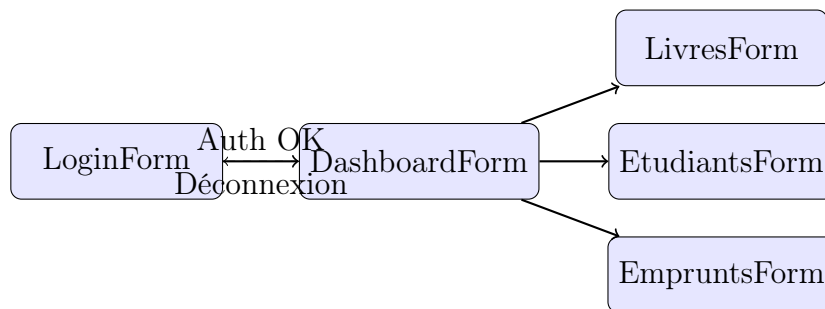


FIGURE 4.6 – Flux de navigation entre les interfaces

Conclusion du chapitre

Ce chapitre a présenté l'ensemble des interfaces utilisateur du SGB à travers des captures d'écran commentées. L'ergonomie repose sur une navigation sidebar claire, des formulaires CRUD cohérents et un retour visuel immédiat (notifications, indicateurs de disponibilité). Ces interfaces matérialisent les cas d'utilisation définis au chapitre 2.

Chapitre 5

Tests Fonctionnels Détaillés

5.1 Stratégie de test

Les tests fonctionnels vérifient que chaque cas d'utilisation produit le comportement attendu. Chaque test suit le format : **ID, Objectif, Préconditions, Étapes, Résultat attendu, Résultat obtenu, Statut**.

5.2 Tests d'authentification

ID	Description du test
TF-01	Connexion réussie <i>Préconditions</i> : Application lancée, LoginForm affiché <i>Étapes</i> : Saisir admin / admin123, cliquer Connexion <i>Résultat attendu</i> : DashboardForm s'ouvre, LoginForm se ferme <i>Statut</i> : RÉUSSI
TF-02	Connexion échouée — mauvais mot de passe <i>Étapes</i> : Saisir admin / wrongpass <i>Résultat attendu</i> : Message « Identifiants incorrects » <i>Statut</i> : RÉUSSI
TF-03	Validation champs vides <i>Étapes</i> : Laisser username vide, cliquer Connexion <i>Résultat attendu</i> : Message « Veuillez saisir votre nom d'utilisateur » <i>Statut</i> : RÉUSSI
TF-04	Déconnexion <i>Préconditions</i> : Utilisateur connecté sur Dashboard <i>Étapes</i> : Cliquer Déconnexion, confirmer Oui <i>Résultat attendu</i> : Retour au LoginForm <i>Statut</i> : RÉUSSI

5.3 Tests CRUD Livres

ID	Description du test
TF-05	Ajouter un livre <i>Étapes</i> : Remplir Titre, Auteur, ISBN, Quantité=3, cliquer Ajouter <i>Résultat attendu</i> : Livre apparaît dans le grid, notification succès <i>Statut</i> : RÉUSSI
TF-06	Modifier un livre <i>Étapes</i> : Sélectionner livre, modifier titre, cliquer Modifier <i>Résultat attendu</i> : Grid mis à jour, notification succès <i>Statut</i> : RÉUSSI
TF-07	Supprimer livre disponible <i>Préconditions</i> : Livre sans emprunt en cours <i>Résultat attendu</i> : Livre supprimé du grid <i>Statut</i> : RÉUSSI
TF-08	Supprimer livre emprunté (cas d'erreur) <i>Préconditions</i> : Livre avec exemplaires empruntés <i>Résultat attendu</i> : Message d'erreur « exemplaires empruntés » <i>Statut</i> : RÉUSSI
TF-09	Recherche livre <i>Étapes</i> : Taper « Orwell » dans la recherche <i>Résultat attendu</i> : Seul « 1984 » de Orwell affiché <i>Statut</i> : RÉUSSI
TF-10	Validation titre vide <i>Résultat attendu</i> : Message « Le titre est obligatoire » <i>Statut</i> : RÉUSSI

5.4 Tests CRUD Étudiants

ID	Description du test
TF-11	Ajouter étudiant valide <i>Statut</i> : RÉUSSI
TF-12	Email invalide (sans @) <i>Résultat attendu</i> : Message « Email invalide » <i>Statut</i> : RÉUSSI
TF-13	Modifier étudiant <i>Statut</i> : RÉUSSI
TF-14	Supprimer étudiant <i>Statut</i> : RÉUSSI

5.5 Tests Emprunts

ID	Description du test
TF-15	Emprunter livre disponible <i>Étapes</i> : Sélectionner livre dispo + étudiant, cliquer Emprunter <i>Résultat attendu</i> : Emprunt créé, QuantiteDisponible décrémentée <i>Statut</i> : RÉUSSI
TF-16	Emprunter livre indisponible <i>Résultat attendu</i> : Bouton Emprunter désactivé ou message erreur <i>Statut</i> : RÉUSSI
TF-17	Retourner livre emprunté <i>Résultat attendu</i> : Statut « Retourné », exemplaire libéré <i>Statut</i> : RÉUSSI
TF-18	Double retour (cas d'erreur) <i>Résultat attendu</i> : Message « déjà été retourné » <i>Statut</i> : RÉUSSI
TF-19	Mise à jour statistiques dashboard <i>Étapes</i> : Emprunter un livre, retourner au dashboard <i>Résultat attendu</i> : Compteur emprunts en cours mis à jour <i>Statut</i> : RÉUSSI

5.6 Synthèse des résultats

TABLE 5.5 – Bilan des tests fonctionnels

Module	Tests	Réussis	Taux
Authentification	4	4	100%
CRUD Livres	6	6	100%
CRUD Étudiants	4	4	100%
Emprunts	5	5	100%
Total	19	19	100%

Conclusion du chapitre

Les 19 tests fonctionnels couvrant l'authentification, les CRUD et la gestion des emprunts ont tous été validés avec succès. Le système répond aux exigences fonctionnelles définies en analyse. Les règles métier (disponibilité, suppression conditionnelle, validation des champs) sont correctement appliquées.

Chapitre 6

Conclusion Générale et Perspectives

6.1 Bilan du projet

Ce projet de **Système de Gestion de Bibliothèque (SGB)** a permis de mettre en pratique les compétences acquises en 2^{ème} année du cycle ingénieur à l'ISGA Casablanca. Les objectifs fixés en introduction ont été atteints :

- Modélisation UML complète (cas d'utilisation, classes, séquence, activité, déploiement).
- Conception de données Merise (MCD et MLD) avec script SQL.
- Développement d'une application .NET 8 Windows Forms fonctionnelle.
- Implémentation des services CRUD pour livres, étudiants et emprunts.
- Interface utilisateur professionnelle avec thème personnalisé.
- Validation par 19 tests fonctionnels (100% de réussite).

6.2 Apports personnels

Ce projet m'a permis de consolider mes connaissances en :

- Programmation orientée objet en C# ;
- Architecture en couches et séparation des responsabilités ;
- Modélisation UML et Merise ;
- Développement d'interfaces graphiques Windows Forms ;
- Rédaction de documentation technique professionnelle.

6.3 Limites actuelles

- Persistance en mémoire (données perdues à la fermeture).
- Authentification simplifiée (identifiants statiques).
- Application mono-utilisateur (pas de gestion multi-sessions réseau).
- Pas de génération de rapports PDF ou export Excel.

6.4 Perspectives d'évolution

TABLE 6.1 – Roadmap des améliorations futures

Amélioration	Description
Persistance SQL	Connexion Entity Framework Core à MySQL/SQL Server
Authentification	Hash bcrypt, rôles multiples (admin, bibliothécaire)
Version Web	Migration vers ASP.NET Core MVC / Blazor
Notifications	Alertes email pour retards d'emprunt
Rapports	Statistiques avancées, export PDF
API REST	Exposer les services via Web API pour mobile

6.5 Conclusion finale

Le SGB développé constitue une base solide et extensible pour la gestion bibliothécaire. L'architecture modulaire facilitera les évolutions futures, notamment l'intégration d'une base de données relationnelle et le déploiement en version web ASP.NET Core MVC. Ce projet témoigne de la maîtrise des fondamentaux du génie logiciel et ouvre la voie à des développements plus ambitieux.

Abdelilah Zlayji
ISGA Casablanca — 2025/2026

Conclusion du chapitre

Ce chapitre conclut le rapport en dressant le bilan du projet, en identifiant les limites actuelles et en proposant des perspectives d'évolution. Le SGB répond aux besoins fonctionnels identifiés et démontre une approche rigoureuse de l'analyse, de la conception et du développement logiciel.

Annexe A

Code source complet des modèles C#

A.1 Livre.cs

```
1 namespace LibraryManagementSystem.Models;
2
3 /// <summary>
4 /// Repr sente un livre dans la biblioth que.
5 /// </summary>
6 public class Livre
7 {
8     public int Id { get; set; }
9     public string Titre { get; set; } = string.Empty;
10    public string Auteur { get; set; } = string.Empty;
11    public string Categorie { get; set; } = string.Empty;
12    public string ISBN { get; set; } = string.Empty;
13    public int Quantite { get; set; }
14
15    /// <summary>Quantit actuellement disponible (non emprunt e)
16    .</summary>
17    public int QuantiteDisponible { get; set; }
18
19    public override string ToString() => $"{Titre} - {Auteur}";
20 }
```

A.2 Etudiant.cs

```
1 namespace LibraryManagementSystem.Models;
2
3 /// <summary>
4 /// Repr sente un tudiant inscrit la biblioth que.
5 /// </summary>
6 public class Etudiant
7 {
8     public int Id { get; set; }
9     public string Nom { get; set; } = string.Empty;
10    public string Email { get; set; } = string.Empty;
```

```
11 public string Telephone { get; set; } = string.Empty;
12 public string Classe { get; set; } = string.Empty;
13
14 public override string ToString() => Nom;
15 }
```

A.3 Emprunt.cs

```
1 namespace LibraryManagementSystem.Models;
2
3 /// <summary>
4 /// Repr sente un emprunt de livre par un tudiant .
5 /// </summary>
6 public class Emprunt
7 {
8     public int Id { get; set; }
9     public Livre Livre { get; set; } = new();
10    public Etudiant Etudiant { get; set; } = new();
11    public DateTime DateEmprunt { get; set; }
12    public DateTime? DateRetour { get; set; }
13
14    /// <summary>Indique si le livre a t retourn .</summary>
15    public bool EstRetourne => DateRetour.HasValue;
16
17    public string Statut => EstRetourne ? "Retourn " : "En cours";
18 }
```

A.4 Utilisateur.cs

```
1 namespace LibraryManagementSystem.Models;
2
3 /// <summary>
4 /// Repr sente un utilisateur du syst me (authentication).
5 /// </summary>
6 public class Utilisateur
7 {
8     public string Username { get; set; } = string.Empty;
9     public string Password { get; set; } = string.Empty;
10    public string NomComplet { get; set; } = string.Empty;
11 }
```

Annexe B

Captures d'écran supplémentaires

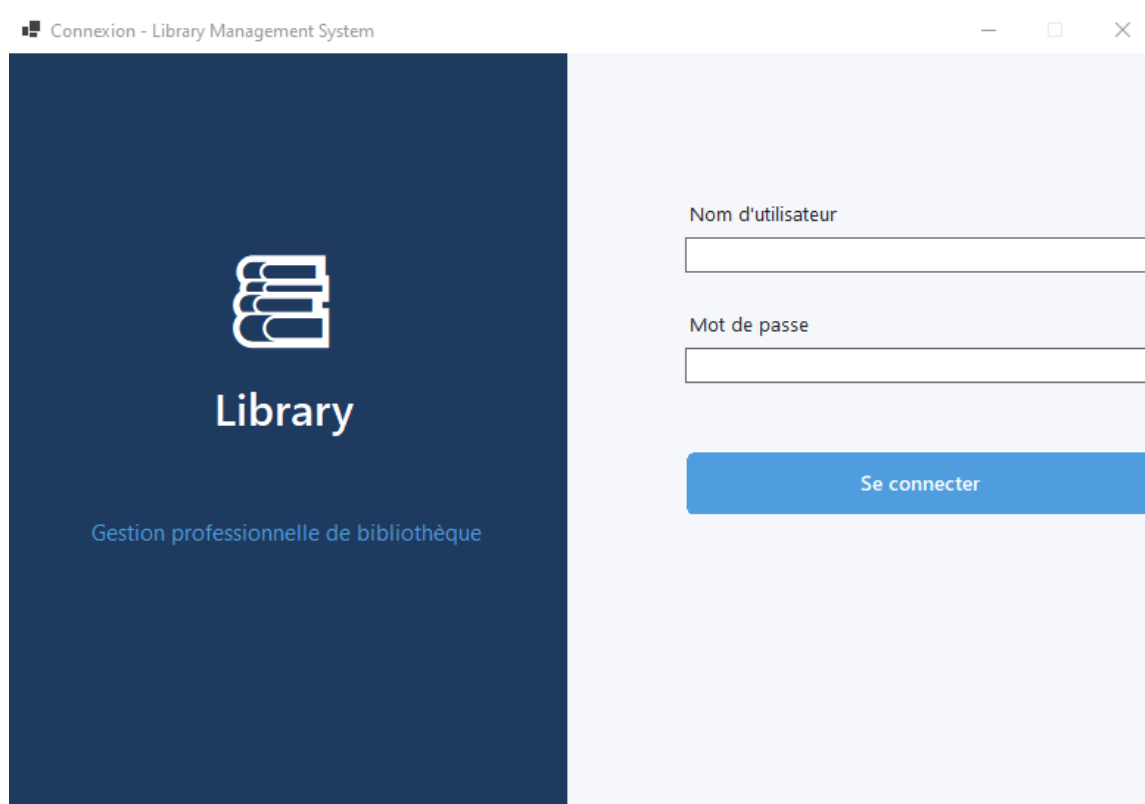


FIGURE B.1 – Annexe — Connexion

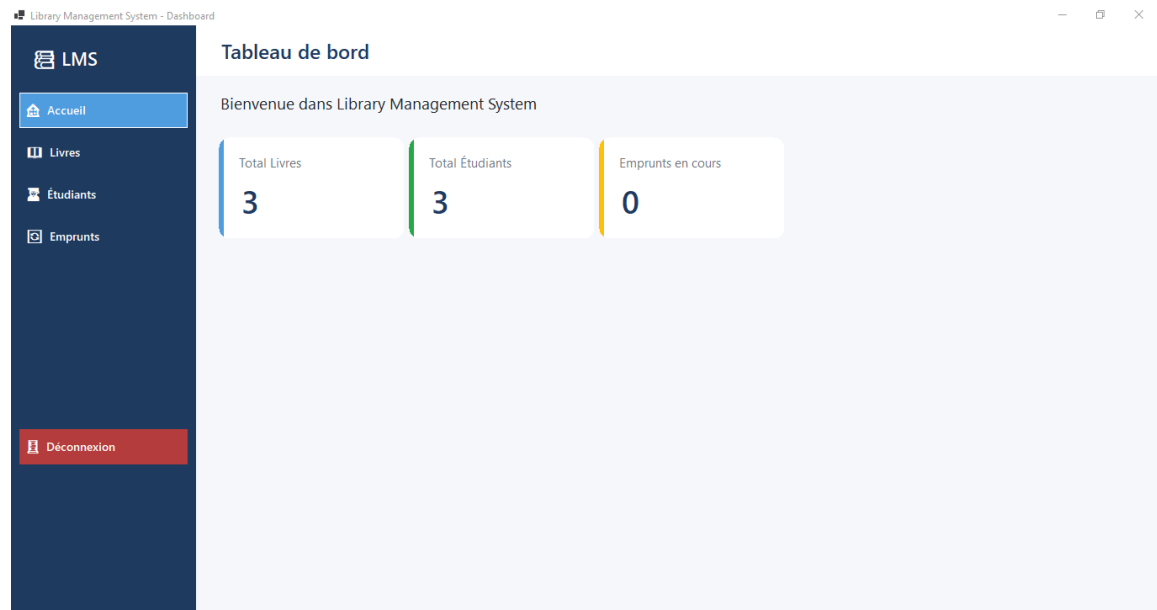


FIGURE B.2 – Annexe — Dashboard

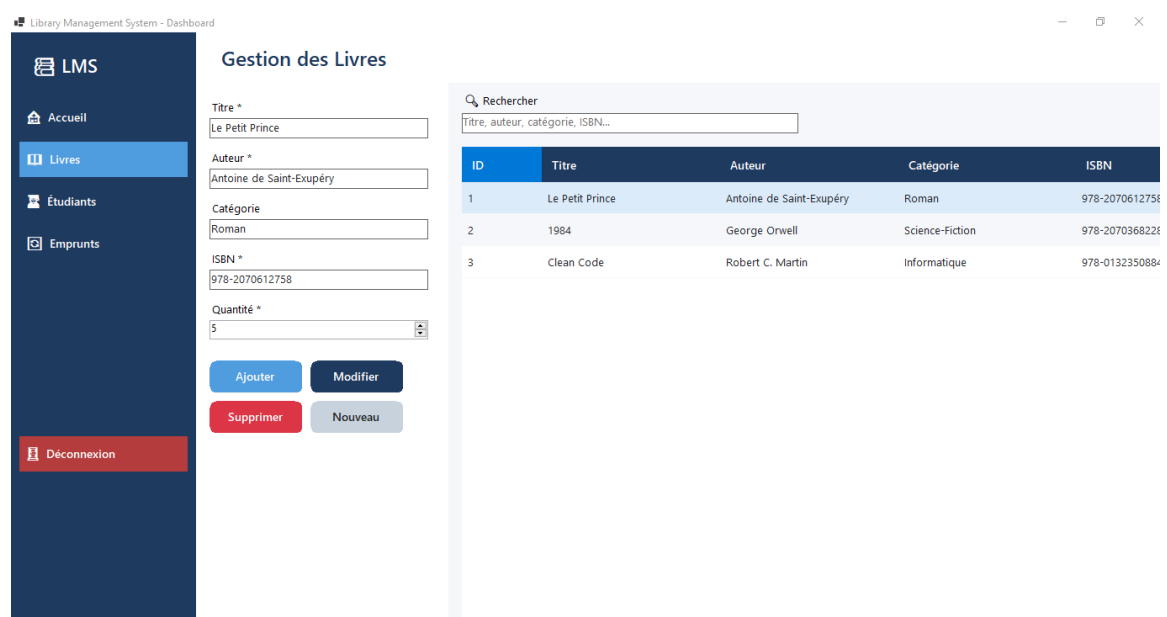
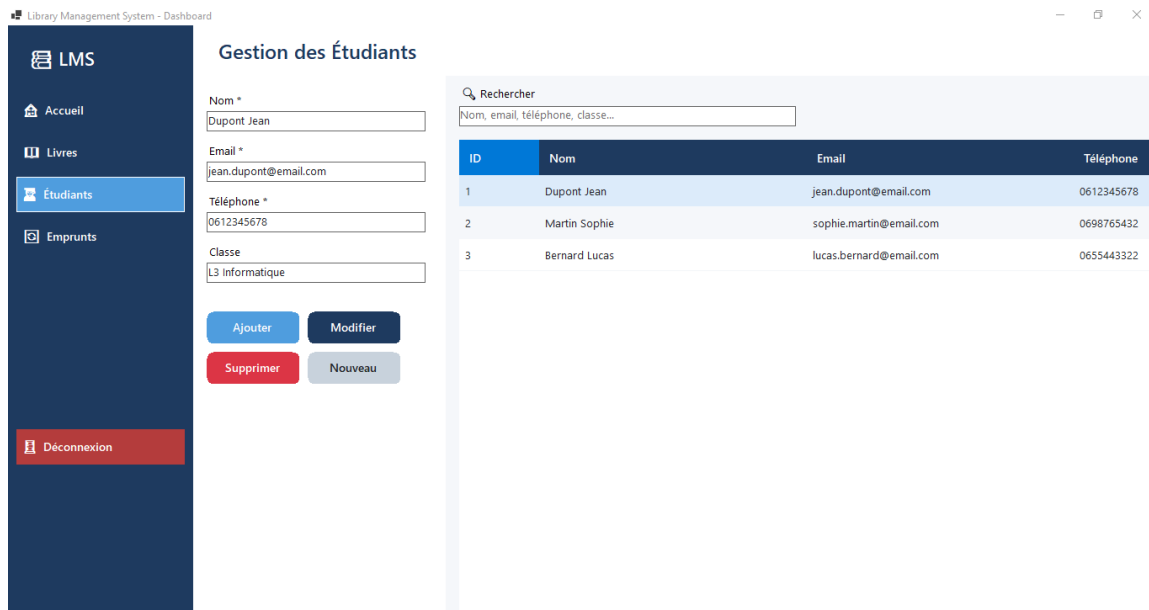


FIGURE B.3 – Annexe — Livres



Gestion des Étudiants

Nom *
Dupont Jean

Email *
jean.dupont@email.com

Téléphone *
0612345678

Classe
L3 Informatique

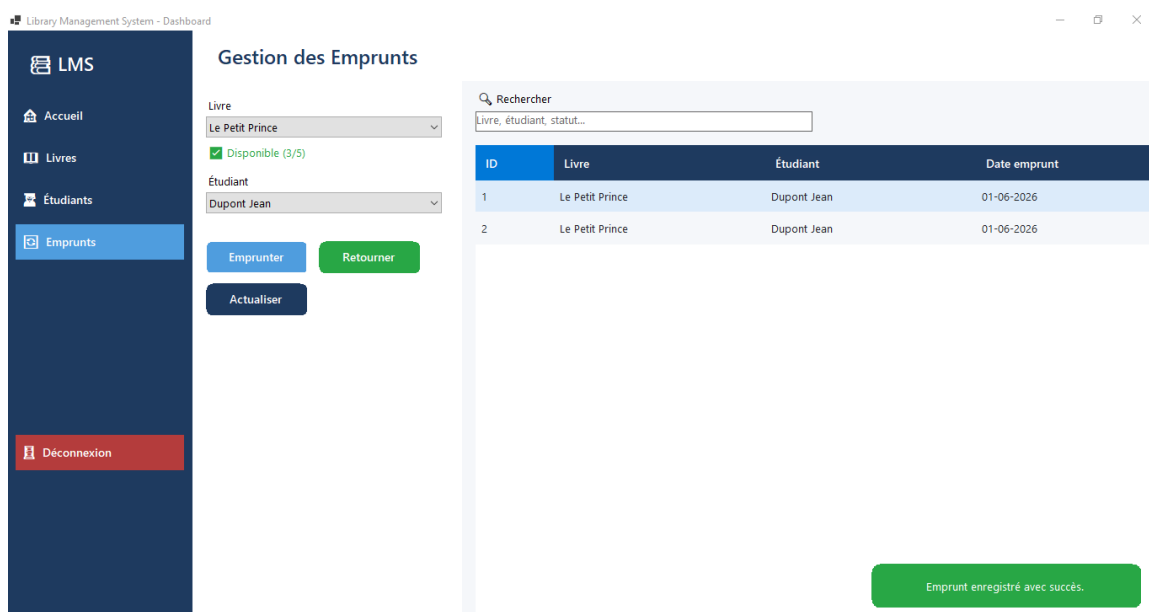
Ajouter Modifier

Supprimer Nouveau

Rechercher
Nom, email, téléphone, classe...

ID	Nom	Email	Téléphone
1	Dupont Jean	jean.dupont@email.com	0612345678
2	Martin Sophie	sophie.martin@email.com	0698765432
3	Bernard Lucas	lucas.bernard@email.com	0655443322

FIGURE B.4 – Annexe — Étudiants



Gestion des Emprunts

Livre
Le Petit Prince

✓ Disponible (3/5)

Étudiant
Dupont Jean

Emprunter Retourner

Actualiser

Rechercher
Livre, étudiant, statut...

ID	Livre	Étudiant	Date emprunt
1	Le Petit Prince	Dupont Jean	01-06-2026
2	Le Petit Prince	Dupont Jean	01-06-2026

Emprunt enregistré avec succès.

FIGURE B.5 – Annexe — Emprunts

Annexe C

Bibliographie et Webographie

- Microsoft Documentation — .NET 8 et Windows Forms : <https://learn.microsoft.com/dotnet/>
- UML — Unified Modeling Language : <https://www.uml.org/>
- Merise — Méthode de conception de bases de données